



**The University of Azad Jammu and Kashmir,
Muzaffarabad**

Name Kamal Ali Akmal

Course Name	Data Structure & Algorithm
Submitted to	Engr. Asma Javaid
Semester	3rd
Session	2024-2028
Roll No	2024-SE-38
Lab No	08
Submission date	08 January 2026

1. What is a Stack?

A **Stack** is a **linear data structure** that follows the **LIFO (Last In, First Out)** principle. This means the last element added to the stack will be the first one to be removed. Think of it like a **stack of plates** – you add plates to the top and remove them from the top.

2. Key Operations in a Stack

Operation	Description
Push	Adds an element to the top of the stack.
Pop	Removes the top element from the stack.
Peek / Top	Returns the top element without removing it.
isEmpty	Checks if the stack contains no elements.
isFull	Checks if the stack has reached its maximum capacity (for fixed-size implementations).

3. Types of Stack Implementations

A. Stack Using Array (Static Implementation)

- Uses a **fixed-size array** to store elements.
- Maintains a **top** pointer (or index) that tracks the topmost element.
- **Advantages:**
 - Simple to implement
 - Memory efficient (no pointer overhead)
- **Disadvantages:**
 - Fixed size (cannot grow beyond initial capacity)
 - Can lead to stack overflow if capacity is exceeded

```

[*] Lab_08_DSA_2024-SE-38.cpp
1  // TASK 01 (Stack Using Array)
2
3  #include <iostream>
4  using namespace std;
5
6  class Stack {
7      int arr[100];
8      int top;
9  public:
10     Stack() {
11         top = -1;
12     }
13
14     bool isEmpty() {
15         return (top == -1);
16     }
17
18     bool isFull() {
19         return (top == 99);
20     }
21
22     void push(int value) {
23         if (isFull()) {
24             cout << "Stack is full! Cannot push " << value << endl;
25             return;
26         }
27         arr[++top] = value;
28     }
29
30     void pop() {
31         if (isEmpty()) {
32             cout << "Stack is empty! Cannot pop" << endl;
33             return;
34         }

```

```

35         top--;
36     }
37
38     int peek() {
39         if (isEmpty()) {
40             cout << "Stack is empty!" << endl;
41             return -1;
42         }
43         return arr[top];
44     }
45 };
46
47 int main() {
48     Stack s;
49
50     // Test basic operations
51     s.push(10);
52     s.push(20);
53     s.push(30);
54
55     cout << "Top element: " << s.peek() << endl; // 30
56
57     s.pop();
58     cout << "After pop, Top element: " << s.peek() << endl; // 20
59
60     s.pop();
61     s.pop();
62
63     cout << "After all pops, Top element: " << s.peek() << endl; // -1
64     cout << "Is stack empty? " << (s.isEmpty() ? "Yes" : "No") << endl; // Yes
65
66     return 0;
67 }

```

```
D:\UNIVRERSITY\3RD SEMES1  ×  +  ∨  
Top element: 30  
After pop, Top element: 20  
Stack is empty!  
After all pops, Top element: -1  
Is stack empty? Yes  
  
-----  
Process exited after 0.231 seconds with return value 0  
Press any key to continue . . . |
```

B. Stack Using Linked List (Dynamic Implementation)

- Uses **dynamic memory allocation** (nodes) to store elements.
- Each node contains:
 - **data:** the element value
 - **next:** pointer to the next node
- The **top** pointer points to the first node (head of the list).
- **Advantages:**
 - No fixed size limit
 - Can grow and shrink as needed
- **Disadvantages:**
 - Extra memory for storing pointers
 - Slightly slower due to dynamic allocation

```

[*) Lab_08_DSA_2024-SE-38.cpp
69 // TASK 02 ( Stack Using Linked List)
70
71 #include <iostream>
72 using namespace std;
73
74 struct Node {
75     int data;
76     Node* next;
77 };
78
79 class Stack {
80     Node* top;
81 public:
82     Stack() {
83         top = nullptr;
84     }
85
86     bool isEmpty() {
87         return (top == nullptr);
88     }
89
90     void push(int value) {
91         Node* temp = new Node();
92         temp->data = value;
93         temp->next = top;
94         top = temp;
95     }
96
97     void pop() {
98         if (isEmpty()) return;
99         Node* temp = top;
100         top = top->next;
101         delete temp;
102     }

```

```

[*) Lab_08_DSA_2024-SE-38.cpp
103
104     int peek() {
105         if (isEmpty()) return -1;
106         return top->data;
107     }
108 };
109
110 int main() {
111     Stack s;
112
113     // Test basic operations
114     s.push(10);
115     s.push(20);
116     s.push(30);
117
118     cout << "Top element: " << s.peek() << endl; // 30
119
120     s.pop();
121     cout << "After pop, Top element: " << s.peek() << endl; // 20
122
123     s.pop();
124     s.pop();
125
126     cout << "After all pops, Top element: " << s.peek() << endl; // -1
127     cout << "Is stack empty? " << (s.isEmpty() ? "Yes" : "No") << endl; // Yes
128
129     return 0;
130 }
131

```

```

D:\UNIVRERSITY\3RD SEMEST
Top element: 30
After pop, Top element: 20
After all pops, Top element: -1
Is stack empty? Yes

-----
Process exited after 0.2395 seconds with return value 0
Press any key to continue . . .

```